



**Ciências
ULisboa**

Privacy and Data Security Project

First Delivery Report of Secure P2P Messaging App

Henrique Alípio	64452
Denis Ungureanu	56307
Pedro Marques	64857

MEI & MSI in Computer Science and Engineering

Introduction

The programming language used for this project is Java, with Maven as the build tool, and the application was developed in Visual Studio and IntelliJ. The user interface was created with JavaFX, and SSL sockets were implemented to ensure secure communication between peers. Version control was managed through GitHub, facilitating collaboration among team members.

The project is structured as follows:

- **CentralServer** – a server that controls users information and generates the truststore.
- **SecureP2PMessagingApp** – the main class of the project where the interface is loaded.
- **P2PServer** – handles incoming messages from peers.
- **P2PClient** – allows users to send messages.

Additionally, three core entities are defined:

- **Conversation** – manages message exchanges between users.
- **Message** – represents individual messages within conversations.
- **User** – identifies each user within the app.

Features

The secure P2P messaging application offers the following features:

- **Direct Messaging:** Enables users to initiate peer-to-peer communication by entering the recipient's username. If that username exists, the central server provides the recipient's IP address and port number to the sender. The sender can then send messages directly to the recipient.
- **Conversation Management and History:** Tracks all messages exchanged in the Conversations tab, with timestamps for chronological ordering.
- **Automated Conversation Handling:** Creates a new conversation thread when a message from a new peer is received.

Security

Application security is implemented through SSL sockets, establishing encrypted channels that guard against interception and tampering. Key management is facilitated using individual keystores, generated by each peer, alongside a truststore, created exclusively by the central server and shared across all peers. The central server also generates its own keystore. Upon joining the network, each peer generates a certificate and imports it into the truststore, enabling secure handshakes between peers when exchanging messages.

This setup achieves:

- **Confidentiality:** Keeps communication private through encryption.
- **Integrity:** Prevents message tampering.
- **Authenticity:** Validates peer identity through certificates.

Interface

Upon initialization, the application prompts the user to enter their login credentials, specifically the username and port number (with the IP address defaulted to localhost). Following successful login, the main menu interface is displayed, presenting several navigation options in the form of buttons:

- **Send Direct Message:** Form for recipient details (username, message).
- **Conversations:** Displays ongoing conversations and chat history with timestamps.
- **Exit:** Closes the application and terminates active connections.

Additional Information

The exception handling mechanism is not implemented in the interface (e.g., same username, same port number, etc.).

Tutorial: Running the Application

BEFORE running the application, make sure that you open the project in your IDE on this directory:

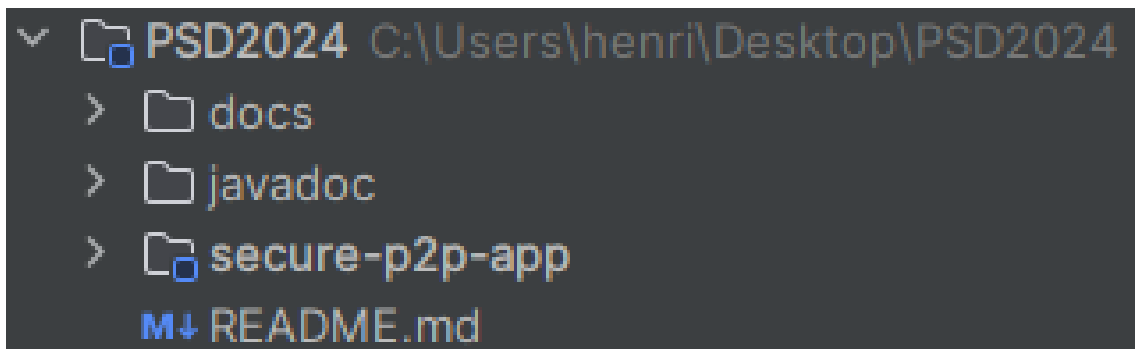


Figure 1: IDE Project Folders

```
// Directory for storing keystore and truststore files
8 usages
public static final String STORE_DIRECTORY = "secure-p2p-app\\src\\main\\java\\com\\psd\\stores\\";
```

Figure 2: EncryptionServer.java - STORES relative path

To run the application, follow these steps:

1. Install Maven
2. Compile the application if there are changes in the source code.

```
mvn clean compile -f "C:\Path\To\Your\Project\secure-p2p-app\pom.xml"
```

3. Run `centralServer`'s main to start the central server.
4. For each peer (user), run `SecureP2PMessagingApp`'s main to start the application using this command line:

```
mvn exec:java -f "C:\Path\To\Your\Project\secure-p2p-app\pom.xml"
```

Conclusion

The project meets the requirements of a peer-to-peer messaging system, achieving a high degree of decentralization with a central server used solely for aggregating peer information. Leveraging SSL for secure communication, JavaFX for a responsive user interface, and a structured application architecture, this platform enables secure, organized, and direct messaging between users.